

Основи на Интернет и мрежовите протоколи. Протоколът HTTP

Предмет	Интернет програмиране
Клас	_____
Специалност	Приложно програмиране
Вид занятие	Нови знания
Име на ученика	_____
Дата	_____

Всеки път, когато отворите сайт, мобилно приложение или онлайн магазин, между вашето устройство и отдалечен сървър се обменят данни по предварително определени правила. Уеб разработчикът трябва да разбира тези правила, за да създава бързи, надеждни и сигурни приложения.

Интернет не е „облак“, който работи самостоятелно. Той е глобална мрежа от свързани мрежи, устройства, маршрутизатори и сървъри, които използват общи протоколи за комуникация.

След този урок ще можете да:

- обяснявате ролята на IP адреса, DNS и мрежовите протоколи;
- разпознавате нивата в модела TCP/IP и ролята им при уеб комуникация;
- анализирате структурата на HTTP заявка и HTTP отговор;
- използвате `curl` и панела **Network** в браузърните инструменти за разработчици, за да наблюдавате HTTP трафик.

2. Основни понятия

2.1. Какво е Интернет?

Интернет е глобална система от взаимно свързани компютърни мрежи. Устройствата в нея обменят данни чрез стандартизирани правила, наречени **мрежови протоколи**.

Важно: Интернет е инфраструктурата и мрежата. **Уебът (World Wide Web)** е една от услугите, които работят върху нея. Други услуги са електронната поща, облачното съхранение, видеоконференциите и др.

2.2. Какво е мрежов протокол?

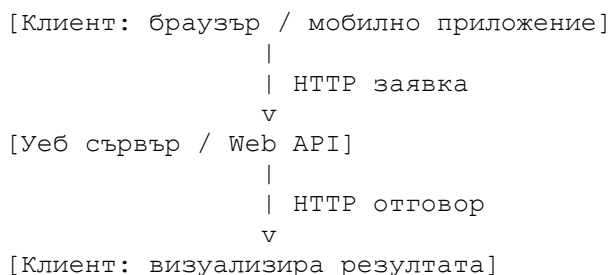
Мрежов протокол е набор от правила, които определят:

1. как изглеждат данните;
2. как се изпращат данните;
3. как получателят разбира и обработва данните;
4. какво се случва при грешка.

Пример: HTTP определя как браузърът да поиска веб страница и как сървърът да върне отговор.

2.3. Клиент и сървър

В веб приложенията най-често се използва моделът **клиент–сървър**.



- **Клиент** — програма, която иска услуга или ресурс. Например Chrome, Firefox, мобилно приложение или JavaScript код.
- **Сървър** — програма/компютър, който приема заявки, обработва ги и връща отговор. Например веб сървър, API сървър или сървър за база данни.
- **Ресурс** — нещо, което клиентът иска: HTML страница, изображение, JSON данни, файл, запис от база данни и др.

3. IP адреси, домейни и DNS

3.1. IP адрес

Всяко устройство, което комуникира в мрежа, трябва да има адрес. **IP адресът** е логически адрес, чрез който пакетите данни могат да достигнат до правилното устройство.

Примери:

- IPv4: 192.168.1.25
- IPv6: 2001:db8:85a3::8a2e:370:7334

IPv4 адресът се състои от четири числа от 0 до 255, разделени с точки. IPv6 е по-нов формат с много по-голям брой възможни адреси.

3.2. Домейн име

Хората по-лесно запомнят имена като `school.bg` или `example.com`, отколкото числови IP адреси. Такова четимо име се нарича **домейн име**.

3.3. DNS — „телефонният указател“ на Интернет

DNS (Domain Name System) е разпределена система, която преобразува домейн име в IP адрес.

```
Потребител въвежда: https://example.com
      |
      v
DNS търсене: „Кой IP адрес отговаря за example.com?“
      |
      v
Получен IP адрес: 93.184.216.34
      |
      v
Браузърът се свързва със сървъра на този адрес
```

Примерна команда за DNS проверка:

```
nslookup example.com
```

Възможен резултат:

```
Name:      example.com
Address: 93.184.216.34
```

Добра практика: Домейнът не винаги има само един IP адрес. Големите сайтове често използват много сървъри, балансиране на натоварването и CDN мрежи.

4. Моделът TCP/IP

За да не се смесват всички мрежови задачи в една система, комуникацията е разделена на слоеве. В практиката на Интернет широко се използва **моделът TCP/IP**.

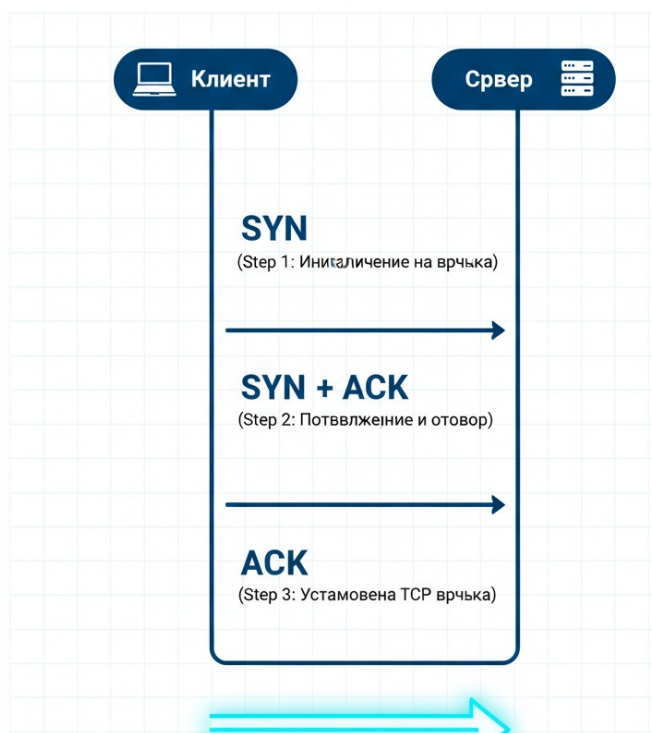
Слой	Основна задача	Примери за технологии/протоколи
Приложен слой	Определя формата и правилата за комуникация между приложенията	HTTP, HTTPS, DNS, SMTP
Транспортен слой	Доставя данните между приложенията на двете устройства	TCP, UDP; портове
Интернет слой	Адресира и маршрутизира пакетите между мрежите	IP, ICMP
Мрежов достъп	Предава данни в локалната мрежа и по физическата среда	Ethernet, Wi-Fi, MAC

4.1. Роля на TCP

TCP (Transmission Control Protocol) е транспортен протокол, който осигурява надеждна доставка на данни. Той:

- установява връзка между две крайни точки;
- разделя данните на части и ги подрежда;
- потвърждава получаването им;
- изпраща отново липсващи или повредени части.

Преди обмен на данни TCP обичайно установява връзка чрез **three-way handshake**:



4.2. Портове

Един IP адрес може да обслужва много приложения. **Портът** посочва конкретната услуга или процес на устройството.

Протокол	Стандартен порт	Предназначение
HTTP	80	Незащитен уеб трафик
HTTPS	443	Защитен HTTP трафик чрез TLS
DNS	53	Преобразуване на домейн имена към IP адреси

Внимание: Портът не е физически „отвор“ в компютъра. Той е логически номер, който операционната система използва, за да насочи данните към правилното приложение.

5. HTTP — протоколът на уеба

5.1. Какво е HTTP?

HTTP (HyperText Transfer Protocol) е приложен протокол за обмен на уеб ресурси. Той работи по модел **заявка–отговор**:

1. клиентът изпраща HTTP заявка;
2. сървърът я обработва;
3. сървърът изпраща HTTP отговор;
4. клиентът визуализира или обработва получения резултат.

Браузър	Уеб сървър
--- GET /products HTTP/1.1 -->	
	обработка
<- HTTP/1.1 200 OK + данни ---	

5.2. HTTP е stateless

HTTP е **протокол без състояние (stateless)**. Това означава, че по подразбиране всяка заявка е самостоятелна: сървърът не е длъжен автоматично да помни предишната заявка на клиента.

За да могат приложенията да „помнят“ потребителя, например при вход в профил или пазарска количка, се използват допълнителни механизми:

- cookies (бисквитки);
- сесии;
- токени, например JWT.

Тези механизми ще бъдат разгледани в следващи уроци.

6. Анатомия на HTTP заявка

HTTP заявката обикновено съдържа:

1. **Start-line (Request line)** — метод, път до ресурса и версия на HTTP;
2. **Headers** — метаданни за заявката;
3. празен ред, който отделя headers от body;
4. **Body** — незадължителни данни, изпращани към сървъра.

Пример: GET заявка

```
GET /products?category=laptops HTTP/1.1
Host: shop.example.com
User-Agent: Mozilla/5.0
```

Accept: text/html

Обяснение:

Част	Значение
GET	HTTP метод; искаме да прочетем ресурс
/products?category=laptops	Пътят до ресурса и параметър на заявката
HTTP/1.1	Използваната версия на HTTP
Host	Домейнът на търсения сървър; задължителен при HTTP/1.1
User-Agent	Информация за клиента, който изпраща заявката
Accept	Формати, които клиентът може да приеме
Празен ред	Край на заглавките; при GET обикновено няма body

Пример: POST заявка с JSON тяло

```
POST /api/students HTTP/1.1
Host: school.example.com
Content-Type: application/json
Content-Length: 42
```

```
{
  "name": "Иван Петров",
  "class": "12A"
}
```

- POST обикновено изпраща данни към сървъра за създаване или обработка.
- Content-Type: application/json казва, че тялото е в JSON формат.
- Content-Length посочва размера на тялото в байтове.

Внимание: В реална HTTP комуникация празният ред между заглавките и тялото е задължителен. Той се предава като `\r\n\r\n` (два последователни края на ред).

7. Анатомия на HTTP отговор

HTTP отговорът съдържа:

1. **Status-line** — версия на HTTP, статус код и текстово описание;
2. **Response headers** — метаданни за отговора;
3. празен ред;
4. **Response body** — върнатото съдържание.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 71
Cache-Control: no-cache
```

```
{
```

```
"id": 12,  
"name": "Иван Петров",  
"class": "12A"  
}
```

Често срещани статус кодове

Код	Значение	Типичен пример
200 OK	Заявката е успешна	Страница или JSON данни са върнати успешно
201 Created	Създаден е нов ресурс	Успешно добавен нов потребител чрез API
301 Moved Permanently	Ресурсът е преместен трайно	Стар URL пренасочва към нов URL
400 Bad Request	Невалидна заявка от клиента	Липсващи/невалидни данни във формуляр
401 Unauthorized	Няма валидна автентикация	Потребителят не е влязъл в системата
403 Forbidden	Достъпът е забранен	Потребител няма нужните права
404 Not Found	Ресурсът не е намерен	Несъществуващ адрес или API маршрут
500 Internal Server Error	Грешка на сървъра	Необработено изключение в приложението

Добра практика: Кодове от 4xx най-често показват проблем в заявката на клиента. Кодове от 5xx показват проблем, възникнал на сървъра.

8. Жизнен цикъл на една уеб заявка

При въвеждане на адреса `https://example.com/products` браузърът извършва приблизително следните стъпки:

- Браузърът анализира URL адреса.
- DNS преобразува `example.com` в IP адрес.
- Установява се връзка със сървъра.
- При HTTPS се извършва TLS договаряне за криптирана връзка.
- Браузърът изпраща HTTP заявка, например `GET /products`.
- Сървърът обработва заявката; възможно е да работи с база данни.
- Сървърът връща HTTP отговор със статус, headers и body.
- Браузърът анализира HTML и изпраща допълнителни заявки за CSS, JavaScript, изображения и шрифтове.
- Страницата се визуализира.

HTTP и HTTPS

- HTTP изпраща данните без криптиране.

- **HTTPS** е HTTP върху защитена TLS връзка. Данните между клиента и сървъра са криптирани при пренос.

Добра практика: В реални приложения използвайте HTTPS. Никога не изпращайте пароли, лични данни или токени през незащитен HTTP.

9. Практическа работа с `curl`

`curl` е инструмент от команден ред за изпращане и изследване на HTTP заявки.

9.1. Обикновена GET заявка

```
curl https://example.com
```

Командата извежда тялото на отговора — обикновено HTML кода на страницата.

9.2. Показване на headers

```
curl -I https://example.com
```

Флагът `-I` изпраща HEAD заявка и показва само headers на отговора.

Примерен резултат:

```
HTTP/2 200
content-type: text/html
content-length: 1256
cache-control: max-age=86000
```

9.3. Подробен мрежов изход

```
curl -v https://example.com
```

Флагът `-v` означава **verbose**. В изхода:

- редовете с `>` са изпратени от клиента;
- редовете с `<` са получени от сървъра.

```
> GET / HTTP/1.1
> Host: example.com
> User-Agent: curl/8.x
> Accept: */*

< HTTP/1.1 200 OK
< Content-Type: text/html; charset=UTF-8
```

9.4. Изпращане на персонализиран header

```
curl -H "X-Student-Id: 12A-15" https://httpbin.org/headers
```

- -H добавя header към заявката.
- `httpbin.org/headers` връща JSON, който показва headers, получени от сървъра.

9.5. Изпращане на JSON с POST

```
curl -X POST https://httpbin.org/post \
  -H "Content-Type: application/json" \
  -d '{"name":"Иван","class":"12A"}'
```

Обяснение:

- -X POST избира HTTP метода POST;
- -H задава типа на изпращаните данни;
- -d подава данните в тялото на заявката.

Внимание: Не използвайте `-k` / `--insecure` в реална среда. Този флаг пропуска проверката на TLS сертификата и може да прикрие сериозен проблем със сигурността.

10. Анализ на заявки с Browser DevTools

Съвременните браузъри позволяват да наблюдаваме реалния HTTP трафик.

Стъпки

1. Отворете сайт в Chrome, Edge или Firefox.
2. Натиснете `F12` или `Ctrl + Shift + I`.
3. Изберете панела **Network**.
4. Презаредете страницата с `Ctrl + R`.
5. Изберете заявка от списъка.
6. Разгледайте секциите **Headers**, **Payload**, **Preview** и **Response**.

Какво да търсим?

Поле в DevTools	Какво показва?
Request URL	Адресът на заявения ресурс
Request Method	Използваният HTTP метод, например GET или POST
Status Code	Резултатът от обработката на заявката
Request Headers	Метаданните, изпратени от браузъра
Response Headers	Метаданните, изпратени от сървъра
Payload	Данните, изпратени в тялото на заявката
Response	Тялото на отговора
Timing	Времето за DNS, свързване, чакане и изтегляне

11. Внимание: чести грешки и добри практики

Чести грешки

1. **Смесване на Интернет и Уеб.** Интернет е мрежата; Уебът е услуга върху нея.
2. **Мислене, че домейнът е същото като IP адрес.** Домейнът е име; DNS го преобразува в IP адрес.
3. **Пренебрегване на HTTP статус кодовете.** Статусът е първата информация, която трябва да проверим при проблем със заявка.
4. **Изпращане на чувствителна информация по HTTP.** Пароли и токени трябва да се предават през HTTPS.
5. **Приемане, че 200 ок винаги означава правилен бизнес резултат.** Отговорът може да е технически успешен, но да съдържа неочаквани или невалидни данни.
6. **Използване на GET за промяна на данни.** GET е предназначен за четене и не трябва да извършва действия с траен ефект.

Добри практики за разработчици

- Проверявайте `Status Code`, `Content-Type` и съдържанието на отговора.
- Използвайте ясни URL адреси и правилни HTTP методи.
- Задавайте коректен `Content-Type` при изпращане на данни.
- Работете с DevTools и логове, вместо да гадаете причината за проблем.
- Използвайте HTTPS във всички реални приложения.
- Не разкривайте пароли, API ключове и токени в код, екранни снимки или публични хранилища.

12. Cheatsheet — речник на новите термини

Термин / синтаксис	Значение	Кратък пример
Интернет	Глобална мрежа от взаимосвързани мрежи	Интернет позволява връзка между клиент и сървър
Протокол	Набор от правила за обмен на данни	HTTP определя формата на уеб заявките
Клиент	Програма, която изпраща заявка	Браузърът е HTTP клиент
Сървър	Програма, която обработва заявка и връща отговор	API сървър връща JSON данни
IP адрес	Логически мрежов адрес	93.184.216.34
DNS	Система за преобразуване на домейн в IP адрес	example.com → IP адрес
TCP	Надежден транспортен протокол	Използва се от HTTP/1.1 и HTTP/2
Порт	Логически номер на услуга в устройство	HTTPS обичайно използва порт 443
HTTP	Протокол за уеб заявка–отговор	GET / HTTP/1.1

Термин / синтаксис	Значение	Кратък пример
HTTPS	HTTP върху TLS криптирана връзка	<code>https://school.example.com</code>
Request line	Първият ред на HTTP заявката	<code>GET /api/users HTTP/1.1</code>
Status line	Първият ред на HTTP отговора	<code>HTTP/1.1 200 OK</code>
Header	Метаданни за заявка или отговор	<code>Content-Type: application/json</code>
Body	Основните данни в заявка/отговор	JSON с данни за потребител
200 OK	Успешно обработена заявка	Страницата е върната успешно
404 Not Found	Търсеният ресурс липсва	Несъществуващ URL адрес
<code>curl -v URL</code>	Подробен изход за HTTP заявка	<code>curl -v https://example.com</code>

13. Самопроверка

Контролни въпроси

1. Каква е ролята на DNS при отваряне на адрес като `https://example.com`?
2. В кой слой на TCP/IP модела се намира HTTP и защо?
3. Кои са четирите основни части на HTTP заявката?
4. Каква е разликата между статус код 404 Not Found и 500 Internal Server Error?

Мини практическа задача 1: Анализ на HTTP headers

1. Изпълнете командата:

```
curl -I https://example.com
```

2. Запишете:

- статус кода;
- стойността на Content-Type;
- дали сървърът връща Content-Length;
- каква информация можете да изведете от тези headers.

Насока: Ако се показва `HTTP/2 200`, това означава успешен отговор, предаден по HTTP/2. `Content-Type` показва как клиентът трябва да интерпретира тялото на отговора.

Мини практическа задача 2: Проверка на персонализиран header

1. Изпълнете командата:

```
curl -H "X-Student-Id: 12A-01" https://httpbin.org/headers
```

2. Намерете `x-student-id` в JSON отговора.
3. Обяснете защо сървърът може да прочете тази стойност.

Насока: Header-ите са част от HTTP заявката. Сървърът ги получава заедно с метода, URL пътя и останалите метаданни.

14. Най-важното от урока

- Интернет свързва мрежи и устройства; протоколите определят правилата за обмен на данни.
- DNS превежда домейн имена към IP адреси.
- TCP/IP моделът разделя мрежовата комуникация на слоеве с различни отговорности.
- HTTP работи чрез заявка–отговор и е протокол без състояние.
- HTTP заявка и отговор имат start-line, headers, празен разделител и по избор body.
- Инструменти като `curl` и Browser DevTools помагат да виждаме действителната комуникация между клиента и сървъра.